Stylinger Product Specification (Extracted from JS)

Document Version: v2.0 — Merchant-First Edition

Author: Zeeshan Vahora, Founder

Organisation: Navrachana University, Vadodara

Status: Pre-Development | Concept & Planning Stage

Last Updated: June 2026

Architecture: Single App, Dual Mode — Modular Design

Primary Focus: Merchant Module (Phase 1 Priority)

Key Design Change: User wardrobe auto-builds from purchases; manual upload optional

1. Executive Summary

Stylinger is an AI-powered fashion platform that acts as the technology layer between India's unorganised offline clothing retailers and their customers. The platform operates through a single mobile application with two modules Merchant Mode and Customer Mode which are architected as completely independent modules but share a common backend, database, and AI engine.

The core mechanic is a QR code placed at a merchant's store entrance. When a customer scans it, Stylinger connects the merchant's live inventory with the customer's personal AI stylist module showing them exactly which items in that store suit their body type and skin tone, displayed on their personal avatar. Simultaneously, every purchase auto-updates the merchant's stock and builds the customer's digital wardrobe.

2. The Key Strategic Decisions

2.1 Merchant-First Development Priority

Phase 1 development focuses primarily on the Merchant Module. The merchant's pain points as in , no inventory tracking, no analytics, no trend intelligence are immediately solvable and have standalone value even before a single customer uses the customer side. This mirrors the UPI adoption model: merchants adopt first because the tool helps them directly, not just their customers.

2.2 Auto-Wardrobe — The New Core Design Decision

The previous design required users to manually photograph and upload their entire wardrobe before using Stylinger. This created a high-friction onboarding barrier. The updated design solves this:

- Manual wardrobe upload is now OPTIONAL, not required to use Stylinger.

- Every item purchased through Stylinger (via QR in-store) is automatically added to the user's digital wardrobe.
- Over time, the wardrobe builds itself through normal shopping behavior zero extra effort from the user.
- Users who want faster personalization can still upload their existing wardrobe manually.
- This eliminates the cold-start problem that would have caused most users to drop off during onboarding.

2.3 Single App, Dual Mode — Modular Architecture

Stylinger is one application, not two. When a user opens the app, they choose their mode Customer or Merchant. Each mode renders a completely different UI but shares the same underlying infrastructure. Every feature, screen, and business logic unit is built as an independent module with clearly defined inputs, outputs, and dependencies.

This ensures:

- Bug fixes in one module cannot break another module.
- New features can be added to either mode without touching the other.
- Future white-labelling (for brands like Pantaloons or Zudio) requires only swapping the UI layer.
- Individual modules can be turned on or off per merchant or per user without rebuilding the app.

3. Complete Module Map

MERCHANT MODULES (M) — Primary Phase 1 Focus

M1: Merchant Onboarding & Store Setup

M2: Inventory Management System

M3: QR Code Generation & Management

M4: Sales Analytics Dashboard

M5: AI Trend Intelligence Engine

M6: Payments & Razorpay Integration

M7: Merchant Notifications & Alerts

CUSTOMER/USER MODULES (U)

U1: User Onboarding & Profile Setup

U2: AI Body, Face & Skin Analysis

U3: Digital Wardrobe (Auto + Optional Manual)

U4: Daily Outfit Planner

U5: In-Store AI Styling (QR Scan Experience)

U6: Purchase History & Spend Tracker

SHARED / CROSS-APP MODULES (S)

S1: Authentication & Account Management

S2: Purchase Flow & Auto-Sync Engine

S3: AI Recommendation & Styling Engine

S4: Push Notification System

S5: Search & Discovery

S6: App Settings & Preferences

4. App Architecture Overview

The following describes how the single app is structured at the code and infrastructure level:

Gateway ↓

Authentication Middleware ↓

Controllers ↓

Services ↓

Repositories ↓

Database

4.1 Frontend Layer (Flutter)

- Single Flutter codebase compiles to both Android and iOS.
- Mode Router: On login, the app checks if the account is Merchant or Customer and routes to the correct module tree.
- Each module is a separate Flutter package within the monorepo M2 (Inventory) is imported independently of U4 (Daily Outfit Planner).
- Shared UI components (buttons, cards, color scheme) live in a 'stylinger_ui' package used by both modes.

4.2 Backend Layer (Node.js / Python)

- REST API with versioning (/api/v1/merchant/, /api/v1/user/, /api/v1/shared/).
- Each module has its own controller, service, and route file. No cross-module function calls allowed â only API calls.
- AI inference runs as separate Python microservices called by the main Node.js API.
- Razorpay webhook handler lives in a dedicated S2 service, isolated from all other modules.

4.3 Database Layer

- PostgreSQL for relational data: users, merchants, inventory, purchases, wardrobe items.
- Redis for real-time operations: stock count cache, QR session data, live inventory sync.
- Cloudinary or S3 for media: product photos, user wardrobe photos, avatar images.

Purpose

Gets a merchant from zero to a live, QR-enabled store on Stylinger. This module runs once per merchant account and handles all setup required before any other merchant module can function.

Onboarding Flow

Key Data Captured

Store ID: Auto-generated unique identifier (STY-XXXX)

Store Category: Men's / Women's / Both / Kids / All categories

Location: GPS coordinates + manual address (for nearby store discovery)

Operating Hours: Optional — used to warn customers if store is closed

Payment Methods: Declared methods: cash, UPI, card (shown on QR scan screen)

Purpose

The core merchant value proposition. Replaces paper registers, WhatsApp notes, and memory-based inventory tracking with a real-time, auto-updating digital system. This module has standalone value independent of any customer-facing feature.

Item Upload Process

Real-Time Stock Management

- Every purchase made through Stylinger auto-decrements the relevant size count instantly.
- Merchant can manually adjust stock at any time (for non-Stylinger sales, returns, or damaged stock).

- Recalculation of inventory: when a merchant manually edits a count, the system recalculates all dependent analytics (sell-through rate, days of stock remaining).
- Bulk upload supported via CSV for merchants with large catalogues.

Low Stock Alert Rules

Alert Threshold: Configurable per merchant. Default: alert when any size drops to 2 pieces.

Alert Channel: Push notification to merchant's device + in-app dashboard badge.

Alert Content: 'Item: Blue Slim-Fit Shirt — Size M has only 2 pieces left. Reorder?'

Out of Stock: Item automatically marked as unavailable and hidden from customer QR scan.

Restock Action: Merchant taps 'Restock', enters new quantity. Item goes live again instantly.

Proof of Sale Log

- Every sale through Stylinger is logged with: timestamp, item name, size, price, customer ID (anonymised).
- Merchant can export sale logs as PDF or CSV for accounting and GST purposes.
- Returns and exchanges are logged separately and linked to original sale record.

Purpose

The physical-digital bridge that is Stylinger's core USP. Each store gets a unique QR code that connects a customer's scan to that store's live inventory and the customer's personal AI stylist in real time.

QR Code Specifications

Format: QR Code Version 3, Error Correction Level H (30% damage tolerance)

Encoded Data: Stylinger deep link: stylinger://store/{store_id}?session={token}

Session Token: Rotates every 24 hours for security. QR code visual stays the same.

Print Formats: A4 full page, A5 half page, small sticker (10cm x 10cm) — all downloadable as PDF

Display Location: Store entrance, billing counter, individual product tags (future)

Branding: QR includes Stylinger logo and 'Scan for AI Style Advice' text below

QR Scan Flow — What Happens in the Backend

QR Lifecycle -

Generated ↓

Printed ↓

Scanned ↓

Session Created ↓

Purchase ↓

Expired

Multiple Store Support

- A merchant with multiple store locations gets a unique QR per location.
- All locations managed from a single merchant account with a location switcher.
- Analytics are available per-location and aggregated across all locations.

Purpose

Gives small clothing store owners the business intelligence they have never had access to before. Every data point visible in this dashboard comes from actual sales and QR scan activity on Stylinger no manual data entry required beyond the initial inventory upload.

Dashboard Sections

4A — Sales Overview

- Total revenue: today / this week / this month / custom date range.
- Number of items sold in each time period.
- Average transaction value.
- Comparison to previous period (e.g., this week vs last week, +12%).

4B — Best Seller Intelligence

- Items ranked by: total units sold, revenue generated, and scan-to-purchase conversion rate.
- Scan-to-purchase rate shows which items get customer attention but fail to convert â signals a pricing or display issue.
- Size distribution: which sizes are selling fastest per item â tells merchant what to restock.
- Slow movers: items that have been in inventory 30+ days with zero scans â merchant can discount or move them.

4C — Customer Behaviour Analytics

- Average time customers spend browsing after a QR scan before purchasing.
- Anonymised demographic data: approximate age group and gender breakdown of buyers (derived from avatar profile data with consent).
- Peak hours: what time of day most scans and purchases happen â helps with staffing.
- Repeat customer rate: how many customers have scanned this store more than once.

4D — Automated Reports

- Daily summary: sent every night at 9 PM via push notification. Total sales, top item, low stock alerts.
- Weekly report: Monday morning. Sales vs previous week, best seller, trend comparison.
- Monthly report: Revenue breakdown, growth metrics, restocking recommendations.

- Quarterly report: Seasonal analysis â what sold in which months.
- Yearly report: Full business intelligence summary. Export as PDF for accountant/GST filing.
- All reports are in 3 categories as specified: Trends, Sales, Inventory.

Revenue -

- Gross Margin
- Average Basket Size
- Average Order Value
- Customer Lifetime Value
- Sell Through Rate
- Inventory Turnover
- Return Rate
- Dead Stock %

Purpose

Gives small Vadodara and Surat shopkeepers the same trend forecasting intelligence that Zara's central buying team uses for free. Tells merchants what to stock before it sells out, not after.

Data Sources for Trend Intelligence

- Stylinger platform data: what items are being scanned and purchased across all merchant stores in the same city.
- Social media signals: trending fashion hashtags and styles from Instagram India (via public API).
- Seasonal patterns: historical Stylinger data on which styles peak in which months for the merchant's specific city.
- Category-specific trends: what is trending for the merchant's store type (e.g., Men's ethnic vs Women's western).

Trend Reports — Content

Current Trending: 'Oversized linen shirts in earthy tones are currently trending in Vadodara. Your stock: 0 units.'

Upcoming Trend: 'Floral prints historically peak in your area 3 weeks before Navratri. Consider stocking by early September.'

What to Restock: 'Your Black Slim-Fit Jeans (Size 32) have sold out. This item sold 8 units last month. Restock recommended.'

What to Discount: 'Your stock of Pastel Polo Shirts has been unsold for 45 days. Recommend 20% discount to clear.'

Competitor Insight: 'Nearby stores with similar inventory have 40% higher QR scan rates for kurtas this week.'

Report Frequency

- Live market trend: real-time dashboard widget showing top 5 trending categories in the merchant's city right now.
- Weekly trend report: delivered every Monday morning as a push notification summary.
- Seasonal alert: 4-6 weeks before major festivals (Navratri, Diwali, Eid, Christmas) with specific stocking recommendations.

Purpose

Enables merchants to collect payment directly through the Stylinger app when a customer makes a purchase after a QR scan. Razorpay is India's most widely used payment gateway, supporting UPI, cards, net banking, and wallets.

Payment Flow

Settlement & Payout

- Razorpay settles merchant payouts on T+2 basis (industry standard).
- Stylinger commission (Phase 2, post-launch): 2-5% deducted before settlement.
- Merchant sees payout history and pending settlements in the M6 dashboard section.
- GST invoicing: Stylinger generates GST-compliant receipts for all digital transactions.

API Integration with Other Systems

- Stylinger exposes a webhook API for merchants who already use billing software (e.g., Marg ERP, Vyapar, Zoho Books).
- When a Stylinger sale happens, the webhook pushes sale data to the merchant's existing billing software automatically.
- In future phases, Stylinger's own commerce platform (MIMP â Merchant Integrated Management Platform) will replace this, offering a full unified dashboard.

Notification Types

New Sale Alert: Instant push when a Stylinger-facilitated purchase is made. Includes item, size, amount.

Low Stock Alert: When any item drops below merchant's configured threshold (default: 2 pieces).

Out of Stock Alert: Item has reached 0 — automatically hidden from customer QR scan.

Daily Sales Summary: 9 PM nightly. Today's revenue, items sold, top performer.

Trend Alert: When a new trending style is identified that matches the merchant's catalogue category.

New QR Scan: Optional: notify merchant when customer scans the QR (useful for small stores to prepare assistance).

Report Ready: When weekly, monthly, or quarterly report is generated and ready to view.

Restock Reminder: If merchant acknowledged a low stock alert but hasn't restocked within 48 hours.

Purpose

Gets a customer from first app open to a working personal AI stylist profile. With the new auto-wardrobe design, onboarding is significantly faster â the only required step is the photo upload for body and skin analysis.

Onboarding Flow — Redesigned for Speed

Preferences Captured During Onboarding

- Style preference: Casual / Smart Casual / Formal / Mix
- used to bias recommendations.
- Budget range: general preferred price range per item used as default filter in stores.
- Gender preference for clothing recommendations.
- Occasion filters most relevant to them: Daily / College / Office / Special Events.

Purpose

The intelligence core of the user side. Converts a customer's photos into structured data that drives all styling recommendations. Built using MediaPipe for geometry and colour science for skin analysis.

Body Shape Analysis

Tool Used: Google MediaPipe Pose — 33 body landmarks extracted from full-body photo

Measurements: Shoulder width, hip width, waist estimate, torso length, leg length ratio

Classification: Rectangle / Inverted Triangle / Pear / Hourglass / Apple

Update Frequency: Re-runs if user uploads new photos. Manual re-trigger available in profile settings.

Face Shape Analysis

Tool Used: MediaPipe Face Mesh — 468 facial landmarks extracted from selfie

Measurements: Face width-to-length ratio, jaw width, forehead width, cheekbone width

Classification: Oval / Round / Square / Heart / Oblong / Triangle

Purpose: Drives collar and neckline recommendations (V-neck for round face, boat neck for square, etc.)

Skin Tone & Undertone Analysis

Tool Used: Colour sampling in LAB colour space from detected facial skin region

Scale: Google Monk Skin Tone Scale (MST) — 10 points, built for Indian and global diversity

Undertone: Warm / Cool / Neutral — determined from hue bias in LAB space

Purpose: Drives colour recommendation engine — which colours, patterns, and contrasts flatter this person

Purpose

Stores the customer's personal clothing collection in digital form. Powers the Daily Outfit Planner and the in-store 'pair with what you own' feature. KEY CHANGE: wardrobe now builds automatically through purchases â manual upload is optional.

Three Ways Items Enter the Wardrobe

AUTO Purchase through Stylinger QR:

- Customer buys any item through a Stylinger in-store QR scan.
- S2 (Purchase Flow & Auto-Sync) module automatically creates a wardrobe entry.
- Item photo, colour, type, fit, and formality all pulled from the merchant's inventory data â no customer effort.
- Customer sees the new item appear in their wardrobe with a confirmation notification.

MANUAL -Customer uploads from home:

- Customer photographs any existing item they own.
- AI auto-tags it (category, colour, pattern, fit, formality).
- Customer reviews tags and corrects any errors.
- This is optional but recommended for a richer Daily Outfit Planner experience.

QUICK ADD Text/category entry (no photo):

- Customer selects category (e.g., White Shirt, Blue Jeans) without uploading a photo.
- Avatar shows a generic version of the item for outfit planning purposes.
- Faster to add but less accurate for visual outfit display.

Wardrobe Organization

- Items organised by category: Tops / Bottoms / Footwear / Accessories / Outerwear / Ethnic.
- Filter by colour, formality, occasion, season.
- 'Unworn' tag: items not suggested in any outfit for 30+ days are flagged â prompts customer to consider donating or selling.
- 'Recently bought' section shows the last 5 Stylinger purchases.

Purpose — The Core Retention Driver

The single feature that makes Stylinger a daily habit rather than an occasional shopping tool. Every morning, the customer opens the app and gets a complete, ready-to-wear outfit suggestion from their wardrobe. No decisions required.

How the Daily Outfit is Generated

Daily Planner Features

- Occasion input: customer taps today's occasion before seeing the outfit. Options: College / Office / Casual / Wedding / Date / Gym / Other.
- Weather integration (future): connects to weather API and avoids suggesting heavy fabrics on hot days or light fabrics in winter.
- Save outfit: customer can save a combination as a favourite for repeated use.
- Calendar view: see what outfit was worn on which past date (feeds from saved outfits).
- 'Buy what's missing' prompt: if the AI detects the outfit would be significantly better with one more item (e.g., a belt or formal shoes the customer doesn't own), it surfaces nearby stores that have it.

Purpose

The merchant-facing feature seen from the customer's side. When a customer scans a QR at a Stylinger-enabled store, they get a personalised, filtered view of that store's live inventory â displayed on their own avatar. This is Stylinger's core USP and the feature that makes the offline-first layer real.

Customer Experience After Scan

- Store profile shown: name, photo, category, available payment methods.
- AI immediately starts matching store inventory against customer's profile.
- Recommendations displayed in ranked order: best match first.
- Each item shown on customer's personal avatar â wearing that exact item.
- 'Pair with what I own': customer can tap any item to see it combined with their existing wardrobe items.
- Budget filter: customer sets a price ceiling and only items within budget are shown.
- Honest mode: if fewer than 3 items in the store genuinely suit the customer, the app says so. 'Only 2 items match your style profile here. Want to see nearby stores?'
- Nearby store discovery: customer can find other Stylinger-enabled stores near them if the current store doesn't match.

The Merchant Counterpart Fear — Addressed

A key merchant concern is: 'If your app tells my customers nothing suits them, why would I participate?' The answer is built into the design:

- Honest no-match actually builds trust. Customers who trust the app's honesty are more likely to buy when it does recommend something.
- The merchant gets QR scan analytics regardless â they see which customers came, what they looked at, and whether they bought elsewhere.
- Merchants with well-curated inventory that genuinely matches a broad range of customer profiles will naturally surface higher in nearby store discovery.
- This creates a healthy incentive: merchants who stock diverse, well-photographed inventory get more QR scans and sales.

Features

- Complete log of all Stylinger purchases: item, store, date, price, size.
- Monthly and yearly spend tracker â total spent on clothing via Stylinger.
- 'From which stores': breakdown of spend by merchant.
- Re-stock alert: if an item the customer scanned but did not buy comes back in stock at the same store, they are notified.
- Return/exchange tracking: customer-initiated returns are logged and linked to original purchase.

Authentication Flow

- OTP-based login via phone number. No passwords. Reduces friction and support burden.
- New user: OTP â mode selection (Customer or Merchant) â respective onboarding.
- Returning user: OTP â directly to their home screen in their chosen mode.
- Single phone number can have both a Customer and a Merchant account. Mode switch via profile menu.
- JWT token-based session management. Token refreshed every 24 hours. Logout invalidates all tokens.

Purpose — The Critical Connector Module

S2 is the most architecturally important shared module. It is the bridge that connects a customer purchase to every other downstream effect: stock decrement, wardrobe update, analytics update, merchant notification, and receipt generation. All of these happen in one atomic transaction â if any step fails, the whole transaction rolls back to prevent data inconsistency.

S2 Transaction Flow — Atomic

Components

- Clothing Tagger: Vision AI (GPT-4 Vision or Gemini Vision API) analyses clothing photo and returns JSON: { category, colour, pattern, fit, formality, collar_type }.
- Body-Colour Rules Engine: Python rules engine that scores each colour and style against the customer's body shape and skin tone. Deterministic, fast, no API cost.
- Outfit Compatibility Engine: Polyvore-trained compatibility model scores item pairs. Runs on Hugging Face inference endpoint.
- Daily Outfit Generator: Combines rules engine + compatibility engine + repeat-memory to produce the daily ranked outfit list.
- In-Store Ranker: Given store inventory + customer profile, ranks items by combined score (body suitability + colour match + compatibility with owned items + budget).

API Cost Management

- Clothing tagger (vision AI) runs once per item at upload time by merchant. Result cached permanently. No repeated API calls per customer scan.

- Body analysis runs once per customer â at onboarding. Cached in customer profile. Re-runs only if customer uploads new photos.
- Daily outfit generation uses only the local rules engine and compatibility model â zero external API calls per daily generation.
- In-store recommendation uses cached merchant item scores + cached customer profile â minimal compute per scan.

Notification Categories & Controls

Transactional: Sale confirmations, receipts, OTPs. Always on. Cannot be disabled.

Operational: Low stock alerts, restock reminders, report ready. Default on. Merchant can disable.

Marketing: Trend updates, nearby store suggestions, new merchant in area. Default on. Customer can disable.

Digest: Daily summary, weekly report. Default on. Configurable time by user.

Phase 1 (Basic)

- Search by store name or area to find Stylinger-enabled stores near the customer.
- Map view: nearby stores displayed on map with distance and category.
- Filter by store type: Men's / Women's / Both / Kids.

Phase 2 (Advanced)

- Search by item type: 'Find a navy blazer near me' â Stylinger searches all nearby merchant inventories.
- Search by budget: 'Formal shirt under Rs. 500 near Alkapuri' â filters across all stores.
- Style discovery feed: personalised feed showing trending items from nearby stores that suit the customer's specific profile.

5. Tech Stack

Frontend App: Flutter (Dart) — single codebase for Android and iOS

Backend API: Node.js (Express) — REST API with modular controller structure

AI/ML Services: Python (FastAPI) microservices — body analysis, outfit engine, clothing tagger

Primary Database: PostgreSQL — users, merchants, inventory, purchases, wardrobe

Cache / Real-Time: Redis — stock count cache, QR session tokens, live sync

Media Storage: Cloudinary — product images, user photos, avatar assets

Payment Gateway: Razorpay API — UPI, cards, wallets

Push Notifications: Firebase Cloud Messaging (FCM) — Android and iOS

AI Vision API: GPT-4 Vision or Google Gemini Vision — clothing tagging at upload

Body Analysis: Google MediaPipe — local inference, zero API cost

Compatibility Model: Hugging Face Inference API — Polyvore-trained outfit compatibility

Skin Tone Scale: Google Monk Skin Tone Scale — open source, local reference

Hosting: AWS (EC2 for API, RDS for PostgreSQL, ElastiCache for Redis, S3 backup)

CI/CD: GitHub Actions — automated test and deploy pipeline per module

6. Database Schema — Key Tables

6.1 Core Tables Overview

merchants: merchant_id, store_name, owner_name, phone, email, gst, location, category, qr_code, status, created_at

users: user_id, phone, name, body_shape, face_shape, skin_tone_mst, undertone, avatar_url, style_pref, budget_range, created_at

inventory_items: item_id, merchant_id, name, category, colour, pattern, fit, formality, mrp, price, photo_urls, ai_tags, created_at

inventory_sizes: size_id, item_id, size_label, quantity, is_available

qr_sessions: session_id, merchant_id, user_id, scan_time, expires_at, purchased (boolean)

orders: order_id, user_id, merchant_id, item_id, size, price, payment_method, razorpay_payment_id, status, created_at

wardrobe_items: wardrobe_id, user_id, item_id (nullable), category, colour, fit, formality, photo_url, source (auto/manual/quick), added_at

outfit_logs: log_id, user_id, outfit_combination (JSON), occasion, date_worn, was_purchased

sale_analytics: analytic_id, merchant_id, item_id, date, units_sold, revenue, scan_count

notifications: notif_id, recipient_id, type, title, body, is_read, created_at

7. Critical Data Flows

7.1 Purchase → Auto-Wardrobe Flow

This is the most important cross-module data flow in the entire application:

7.2 QR Scan → Recommendation Flow

8. Phase 1 MVP — What Gets Built First

The MVP is the smallest version of Stylinger that can be put in front of a real merchant and a real customer in Vadodara and produce genuine value for both. Not everything in this document is in the MVP.

MVP SCOPE — Merchant Side

MVP SCOPE — Customer Side

MVP SCOPE — Shared

Recommendation: For the SSIP beta, use GPT-4 Vision API for clothing tagging (fast to implement) and the Python rules engine for outfit recommendations (no model training required). Replace with trained models in Phase 2 once you have real user data to train on.

9. Modular Development Rules

To ensure bugs in one module cannot cascade to others, the following rules must be enforced throughout development:

No direct function calls between modules.

- Module M2 (Inventory) never calls a function inside U3 (Wardrobe) directly.
- All cross-module communication goes through the API layer. M2 calls POST /api/v1/user/wardrobe/ â□□ it does not import U3 code.

Each module has its own error handling.

- If M4 (Analytics) fails to update, it throws an error to S2 â□□ but S2 has already completed steps C, D, E. M4 failure logs to a retry queue, never rolls back the sale.
- Only S2's atomic transaction (steps A-D) can trigger a full rollback.

Each module has its own test suite.

- M2 tests run without any user modules present. U4 tests run without any merchant modules present.
- Integration tests for S2 test the full cross-module flow in isolation from production.

Feature flags per module.

- Any module can be disabled via a feature flag in the admin panel without redeploying the app.
- Example: M5 (Trend Intelligence) can be turned off for all merchants while a bug is fixed, without affecting M2 or M4.

Version each module's API independently.

- M2 API at /api/v1/merchant/inventory/ can be updated to v2 without touching /api/v1/user/wardrobe/.
- Old versions stay live until all clients have migrated.

10. Post-SSIP Roadmap

Phase 1 — MVP: M1, M2, M3, M6, U1, U2, U3 (auto), U5, S1, S2, S3 (rules only). 10 merchants in Vadodara. 1,000 first users.

Phase 2 — Growth: M4 (full analytics), M5 (trend intelligence), U4 (daily planner), M7 (full notifications), S5 (discovery). Expand to Surat and Ahmedabad. 500 merchants.

Phase 3 — Scale: ML model training on real data. S3 (full AI engine). Mall & franchise store B2B SaaS. Pan-Gujarat.

Phase 4 — Platform: National brand partnerships. Pantaloons, Zudio white-label. Online brand integration (Snitch, Bewakoof). All India.

Phase 5 — Intel: Trend data licensing to manufacturers. Southeast Asia expansion. Fashion intelligence platform.